EEX6335 – Compiler Design
EEX6363 – Compiler Construction

Day School – 3

by

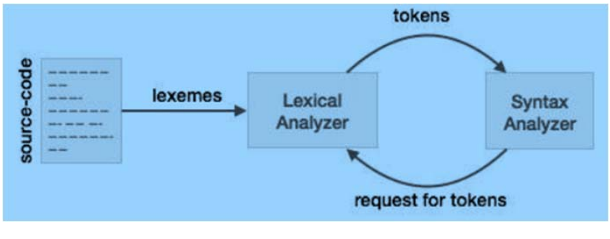
Gehan Anthonys

Bachelor of Science Honours in Engineering
Bachelor of Software Engineering Honours

Department of Electrical and Computer Engineering
Faculty of Engineering
The Open University of Sri Lanka

19 July 2025

Let's consider detail analysis of syntax analyzer:



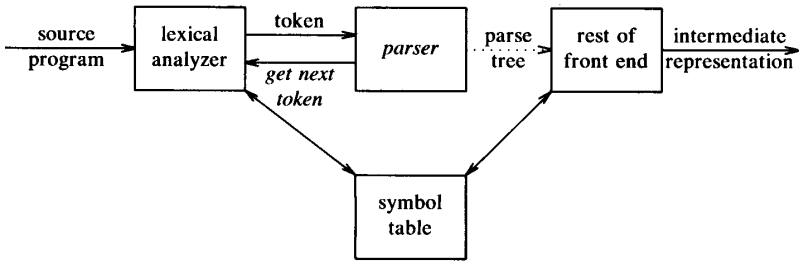
Syntax Analysis

- Context Free Grammars
- Parsing
  - Top-down -- *by left most derivation*
  - Bottom-up (shift-reduce) -- *by right most derivation in reverse*
- Attribute grammars

Syntax (Parser) Analysis

Parser:

If the string is generated from the given grammar, then the parser will generate a parse tree for that given string.



Input: sequence of tokens from lexer.

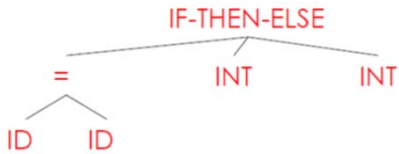
Output: parse tree of the program, but some parsers never produce a parse tree ...

E.g., Let us consider the if-then-else statement of a language:

if x = y then 1 else 2

Parser input → IF ID = ID THEN INT ELSE INT

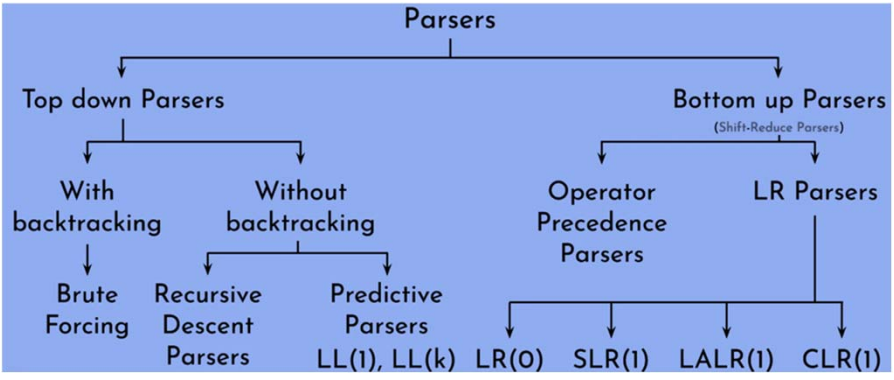
Parser output →



Comparison:

Phase	Input	Output
Lexer	String of characters	String of tokens
Parser	String of tokens	Parse tree

Types of Parsers



The Role of the Parser:

Not **all** strings of tokens are programs;

Thus, the parser must distinguish between valid and invalid strings of tokens. For that, we need:

- A language for describing valid strings of tokens;
- A method for distinguishing valid from invalid strings of tokens.

Programming language constructs have recursive structure:

E.g., An EXPR is

if EXPR then EXPR else EXPR

while EXPR loop EXPR …

Context Free Grammars

-- are a natural notation for this recursive structure.

Context Free Grammars

- A context free grammar consists of terminals, non-terminals, a start symbol, and productions.
- Terminals are the basic symbols from which strings are formed.
- Nonterminals are syntactic variables that denote sets of strings.
- One nonterminal is distinguished as the start symbol.
- The productions of a grammar specify the manner in which the terminal and non-terminals can be combined to form strings.
- A language that can be generated by a grammar is said to be a context-free language.
- A grammar lays down the rules to specify a language as opposed to a recognizer which identifies the strings belonging to a language.

RE relates to Context-Free Grammar

- Symbolically context-free grammar G can be represented as

CFG → G = (N, T (or $\sum$), P, S), where

N : Finite set of variables/non-terminals
T : Alphabet/Finite set of terminals
P : Finite set of rules/productions
S : Start symbol

$S \in N$
$N \cap T = \phi$
Rule : $A \rightarrow \omega$
$A \in N \quad \omega \in (N \cup T)^*$

with * represents a context-free language because we can write a CFG for it.

E.g., $G = (\{S\}, \{a\}, \{S \rightarrow \lambda, S \rightarrow aS\}, S)$

$A \rightarrow \lambda$; *A*-rule, *null*-rule, λ rule.

- *Context-freeness:*

An *A*-rule can be applied whenever *A* occurs in a string, irrespective of the context (that is, non-terminals and terminals around *A*).

- E.g., 1: The grammar with the following productions defines simple arithmetic expressions.

$$
\begin{aligned}
expr &\rightarrow expr\ op\ expr \\
expr &\rightarrow (\ expr\ ) \\
expr &\rightarrow -\ expr \\
expr &\rightarrow \mathbf{id} \\
op &\rightarrow + \\
op &\rightarrow - \\
op &\rightarrow * \\
op &\rightarrow / \\
op &\rightarrow \uparrow
\end{aligned}
$$

In this grammar, the terminal symbols are

id + − ∗ / ↑ ( )

The nonterminal symbols are *expr* and *op*, and *expr* is the start symbol. □

E.g. 2:

$$
\begin{aligned}
EXPR &\rightarrow \text{if EXPR then EXPR else EXPR} \\
&\mid \text{while EXPR loop EXPR} \\
&\mid \text{id}
\end{aligned}
$$

-Non-terminals are written in upper-case.

-Terminals are in lower-case.

-The start symbol is the left-hand side of the first production.

From CFG to Language

- One-step Derivation

$$\bullet uAv \Rightarrow uwv$$
$$A \rightarrow w$$

- w is derivable from v in CFG, if there is a finite sequence of rule applications such that:

$$v \Rightarrow w_1 \Rightarrow \ldots \Rightarrow w_n = w$$

E.g., From the given grammar, $E \rightarrow E{+}E \mid E * E \mid (E) \mid \text{id}$

derive the strings: id ∗ id and id ∗ id + id

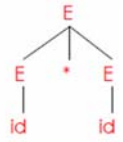
Solution:

We have, $E \rightarrow E+E \mid E*E \mid (E) \mid id$

first consider, id * id

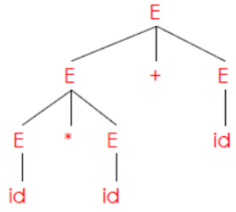
which can be derived as

$$
\begin{aligned}
& E \\
\rightarrow\ & E*E \\
\rightarrow\ & id*E \\
\rightarrow\ & id*id
\end{aligned}
$$



Next, id * id + id can be derived

$$
\begin{aligned}
& E \\
\rightarrow\ & E+E \\
\rightarrow\ & E*E+E \\
\rightarrow\ & id*E+E \\
\rightarrow\ & id*id+E \\
\rightarrow\ & id*id+id
\end{aligned}
$$



Notational Conventions

E.g., Let consider the following grammar:

```
E → EAE | (E) | -E | id
A → + | - | * | / | ↑
```

Derivations

- $E \Rightarrow -E$ is read "E derives -E"
- $E \Rightarrow -E \Rightarrow -(E) = -(\mathbf{id})$ is called a derivation of -(**id**) from E.
- If $A \rightarrow \gamma$ is a production and α and β are arbitrary strings of grammar symbols, we say $\alpha A\beta \Rightarrow \alpha\gamma\beta$.
- If $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \ldots \Rightarrow \alpha_n$, we say α_1 derives α_n.

- $\Rightarrow$ means "derives in one step."
- $\underset{*}{\Rightarrow}$ means "derives in zero or more steps."
  - $\alpha \underset{*}{\Rightarrow} \alpha$
  - if $\alpha \underset{*}{\Rightarrow} \beta$ and $\beta \Rightarrow \gamma$ then $\alpha \underset{*}{\Rightarrow} \gamma$
- $\underset{+}{\Rightarrow}$ means "derives in one or more steps."
- If $S \underset{*}{\Rightarrow} \alpha$, where α may contain nonterminals, then we say that α is a sentential form.

Language, strings, …

- G: grammar, S: start symbol, L(G): the language generated by G.
- Strings in L(G) may contain only terminal symbols of G.
- A string of terminal *w* is said to be in L(G) if and only if $S \overset{+}{\Rightarrow} w$.
- The string *w* is called a sentence of G.
- A language that can be generated by a grammar is said to be a context-free language.
- If two grammars generate the same language, the grammars are said to be equivalent.

Example: $E \rightarrow EAE|(E)|-E|id$

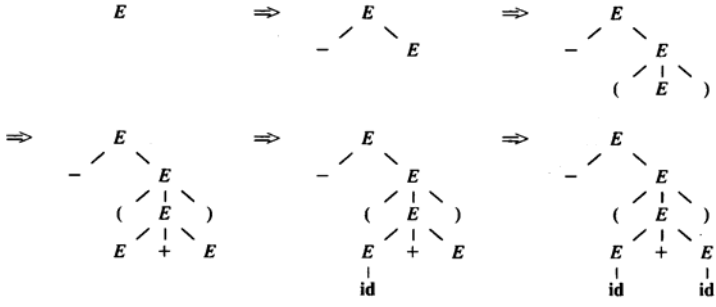
$A \rightarrow +|-|*|/|\uparrow$

The string -(id+id) is a sentence of the above grammar because

$E \Rightarrow -E \Rightarrow \ldots \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$

Therefore, we can write $E \overset{*}{\Rightarrow} -(id+id)$

Now, lets draw the parse tree for this string



This is the parse tree for – (id + id)

Two Parse Trees

$E \rightarrow EAE|(E)|-E|id$

$A \rightarrow + | -| *| /| \uparrow$

Example 4.6. Let us again consider the arithmetic expression grammar The sentence **id+id*id** has the two distinct leftmost derivations:

$$\begin{aligned} E &\Rightarrow E+E \\ &\Rightarrow \mathbf{id}+E \\ &\Rightarrow \mathbf{id}+E*E \\ &\Rightarrow \mathbf{id}+\mathbf{id}*E \\ &\Rightarrow \mathbf{id}+\mathbf{id}*\mathbf{id} \end{aligned} \qquad \begin{aligned} E &\Rightarrow E*E \\ &\Rightarrow E+E*E \\ &\Rightarrow \mathbf{id}+E*E \\ &\Rightarrow \mathbf{id}+\mathbf{id}*E \\ &\Rightarrow \mathbf{id}+\mathbf{id}*\mathbf{id} \end{aligned}$$

with the two corresponding parse trees shown in Fig. 4.4.

-- *Dragon book*

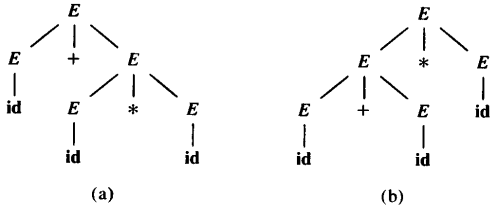


Fig. 4.4. Two parse trees for **id+id*id**.

Ambiguity

A grammar that produces more than one parse tree for some sentence is said to be *ambiguous*.

$$
\begin{aligned}
stmt &\rightarrow \textbf{if}\ expr\ \textbf{then}\ stmt \\
&\mid \textbf{if}\ expr\ \textbf{then}\ stmt\ \textbf{else}\ stmt \\
&\mid \textbf{other}
\end{aligned}
$$

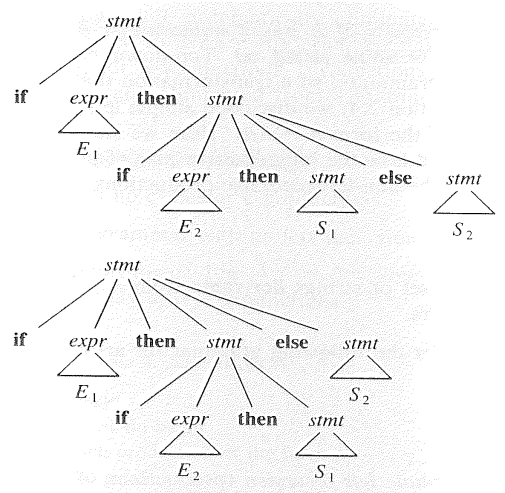


Fig. 4.6. Two parse trees for an ambiguous sentence.

Eliminating Ambiguity

Sometimes an ambiguous grammar can be rewritten to eliminate the ambiguity.

E.g., “match each else with the closest unmatched then”

$$
\begin{aligned}
stmt &\rightarrow matched_stmt \\
&\mid unmatched_stmt \\
matched_stmt &\rightarrow \textbf{if}\ expr\ \textbf{then}\ matched_stmt\ \textbf{else}\ matched_stmt \\
&\mid \textbf{other} \\
unmatched_stmt &\rightarrow \textbf{if}\ expr\ \textbf{then}\ stmt \\
&\mid \textbf{if}\ expr\ \textbf{then}\ matched_stmt\ \textbf{else}\ unmatched_stmt
\end{aligned}
$$

Eliminating Left Recursion

- A grammar is left recursive if it has a nonterminal A such that there is a derivation $A \overset{+}{\Rightarrow} A\alpha$ for some string α.
- $A \rightarrow A\alpha|\beta$ can be replaced by

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A'|\varepsilon$$

- $A \rightarrow A\alpha_1|A\alpha_2|\ldots|A\alpha_m|\beta_1|\beta_2|\ldots|\beta_n|$

$$A \rightarrow \beta_1 A'|\beta_2 A'|\ldots|\beta_n A'|$$

$$A' \rightarrow \alpha_1 A'|\alpha_2 A'|\ldots \alpha_m A'|\varepsilon$$

Algorithm: Eliminating Left Recursion

Input. Grammar G with no cycles or ϵ-productions.

Output. An equivalent grammar with no left recursion.

Method. Apply the algorithm in Fig. 4.7 to G. Note that the resulting non-left-recursive grammar may have ϵ-productions. □

1. Arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2. **for** $i := 1$ **to** n **do begin**
   for $j := 1$ **to** $i-1$ **do begin**
   replace each production of the form $A_i \rightarrow A_j\gamma$
   by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \cdots \mid \delta_k\gamma$,
   where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \cdots \mid \delta_k$ are all the current A_j-productions;
   end
   eliminate the immediate left recursion among the A_i-productions
   end

Examples:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid \mathbf{id}$$

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \epsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \epsilon$$
$$F \rightarrow (E) \mid \mathbf{id}$$

$S \rightarrow Aa|b$

$A \rightarrow Ac|Sd|\varepsilon$

S is left-recursive because

$S \rightarrow Aa \rightarrow Sda$

$A \rightarrow Ac|Aad|bd|\varepsilon$

$S \rightarrow Aa|b$

$A \rightarrow bdA'|A'$

$A' \rightarrow cA'|adA'|\ \varepsilon$

Simplifying Context Free Grammars

- Most of the time, some production rules in CFGs are not useful and are redundant.
- By simplifying CFGs we remove all these redundant productions from a grammar while keeping the transformed grammar equivalent to the original grammar.
- Two grammars are called equivalent if they produce the same language.
- Simplifying CFGs is necessary to later convert them into Normal forms such as CNF, GNF, etc.

Types of redundant productions: **useless, null** and **unit**

Useless productions –

The productions rules that can never take part in derivation of any string.

E.g., Consider the grammar:

S -> abS | abA | abB
A -> cd
B -> aB
C -> dc

S -> abS | abA
A -> cd

Null productions (λ) –

The productions of type 'A -> λ' are called λ productions (also called lambda and null productions).

These productions can only be removed from those grammars that do not generate λ (an empty string).

It is possible for a grammar to contain null productions and yet not produce an empty string.

E.g., Consider the grammar:

S -> ABCd
A -> BC
B -> bB | λ
C -> cC | λ

S -> ABCd | ABd | ACd | BCd | Ad | Bd |Cd | d
A -> BC | B | C
B -> bB | b
C -> cC | c

Unit productions –

The productions of type ‘A -> B’ are called unit productions.

E.g.,

S -> Aa | B
A -> b | B
B -> A | a

S -> Aa
A -> b
B -> a

S -> Aa
A -> b | a
B -> a

S -> Aa
A -> b | a
B -> a | b

S -> Aa | b | a
A -> b | a
B -> a | b

S -> Aa|b|a
A -> b|a

Do you have any questions/clarifications?

… few minutes.

Top-Down Parsing:

- Recursive-descent parsing
- Predictive parsers
- Non-recursive predictive parsing
- FIRST and FOLLOW
- Construction of predictive parsing table
- LL(1) grammars
- Error recovery in predictive parsing

Recursive-Descent Parsing

- Top-down parsing can be viewed as an attempt to find a leftmost derivation for an input string.
- It can also view as an attempt to construct a parse tree for the input string from the root and creating the nodes of the parse tree in preorder.

Grammar:

$S \rightarrow cAd$
$A \rightarrow ab \mid a$

Input string
w = *cad*

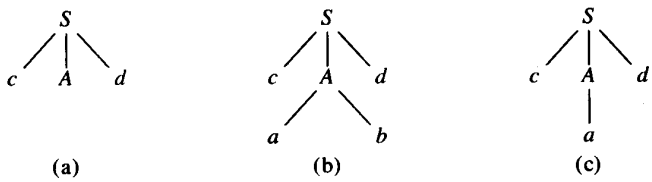


Fig. 4.9. Steps in top-down parse.

Implement of Recursive-Descent Parsing

After removing left-recursion,

$S \rightarrow cAd$
$A \rightarrow ab \mid a$

$S \rightarrow cAd$
$A \rightarrow aA'$
$A' \rightarrow b \mid \varepsilon$

```
S()
{
        if (look_ahead == 'c')
        {
                match('c');
                A();
                match('d');
        }
}
```

```
A()
{
        if (look_ahead == 'a')
        {
                match('a');
                A'();
        }
}
```

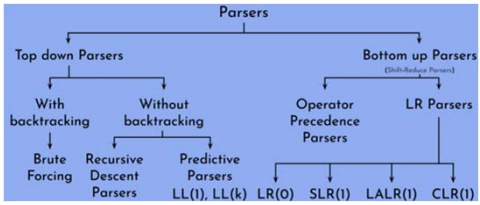
```
A'()
{
        if (look_ahead == 'b')
        {
                match('b');
        }
        else
                return;
}
```

```
match(char i)
{
        if (look_ahead == i)
                look_ahead = getchar();
        else
                printf("Error ");
}
```

```
main()
{
        S();
        if (look_ahead == ' $')
                printf("Parsing Successfully");
}
```

Input string
w = *cad* $

Predictive Parsers



- By carefully writing a grammar, eliminating left recursion, and left factoring the resulting grammar, we can obtain a grammar that can be parsed by a recursive-descent parser that needs <u>no backtracking</u>, i.e., a predictive parser.

$stmt \rightarrow$ **if** $expr$ **then** $stmt$ **else** $stmt$
$\mid$ **while** $expr$ **do** $stmt$
$\mid$ **begin** $stmt_list$ **end**

$S \rightarrow cAd$
$A \rightarrow ab \mid a$

$S \rightarrow cAd$
$A \rightarrow aA'$
$A' \rightarrow b \mid \varepsilon$

Predictive Parsers

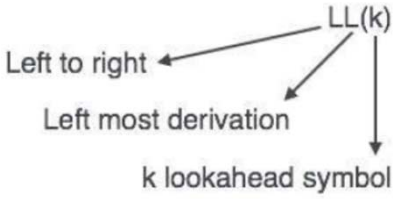
- Recursive-descent parsing is a top-down method of syntax analysis in which we execute a set of recursive procedures to process the input.
- A procedures is associated with each nonterminal of a grammar.
- Predictive parsing is what in which the look-ahead symbol unambiguously determines the procedure selected for each nonterminal.
- The sequence of procedures called in-processing the input implicitly defines a parse tree for the input.

LL(1) Grammars

- A grammar whose parsing table has no multiply-defined entries is said to be LL(1).
- LL Parser accepts LL grammar. LL grammar is a subset of context-free grammar but with some restrictions to get the simplified version, in order to achieve easy implementation.
- LL grammar can be implemented by means of both algorithms namely, recursive-descent or table-driven.

Stands for LL(1):

- First L: scanning from left to right
- Second L: producing a leftmost derivation
- 1: using one input symbol of lookahead at each step to make parsing action decision.



FIRST and FOLLOW sets

- If α is <u>any string</u> of grammar symbols, FIRST(α) is the set of terminals that begin the strings derived from α. If $\alpha \overset{*}{\Rightarrow} \varepsilon$ then ε is also in FIRST(α).
- FOLLOW(A), for non-terminal A, is the set of terminals a that can appear immediately to the right of A in some sentential form, i.e., the set of terminals a such that there exists a derivation of the form $S \Rightarrow \alpha A a \beta$ for some α and β.
- If A can be the rightmost symbol in some sentential form, the $ is in FOLLOW(A).

$S \rightarrow aABC$
$A \rightarrow b$
$B \rightarrow c$
$C \rightarrow d$

FIRST(S): S → a A B C

FIRST(A): A → b

FIRST(B): B → c

FIRST(C): C → d

$S \rightarrow ABC$
$A \rightarrow a$
$B \rightarrow b \mid \varepsilon$
$C \rightarrow c$

S $ → A B C $; A → ε, C → c

FOLLOW(S): { $ }
FOLLOW(A): { b, c }
FOLLOW(B): { c }
FOLLOW(C): { $ }

Properties of LL(1)

- No ambiguous or left recursive grammar can be LL(1).
- Grammar G is LL(1) **iff** whenever $A \rightarrow \alpha|\beta$ are two distinct productions of G and:
  - For no terminal, *a* do both α and β derive strings beginning with *a*.

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \varnothing$$

  - At most one of α and β can derive the empty string.
  - If $\beta \Rightarrow \varepsilon$, the α does not derive any string beginning with a terminal in FOLLOW(A).

$$\text{FIRST}(\alpha\text{FOLLOW}(A)) \cap \text{FIRST}(\beta\text{FOLLOW}(A)) = \varnothing$$

Predictive Parsing Program

Input. A string w and a parsing table M for grammar G.

Output. If w is in $L(G)$, a leftmost derivation of w; otherwise, an error indication.

Method. Initially, the parser is in a configuration in which it has $\$S$ on the stack with S, the start symbol of G on top, and $w\$$ in the input buffer. The program that utilizes the predictive parsing table M to produce a parse for the input is shown in Fig. 4.14. □


```
set ip to point to the first symbol of w$;
repeat
    let X be the top stack symbol and a the symbol pointed to by ip;
    if X is a terminal or $ then
        if X = a then
            pop X from the stack and advance ip
        else error()
    else    /* X is a nonterminal */
        if M[X, a] = X → Y1Y2 · · · Yk then begin
            pop X from the stack;
            push Yk, Yk-1, . . . , Y1 onto the stack, with Y1 on top;
            output the production X → Y1Y2 · · · Yk
        end
        else error()
until X = $    /* stack is empty */
```

Parsing Table, M

Grammar:

$$
\begin{aligned}
E &\rightarrow TE' \\
E' &\rightarrow +TE' \mid \epsilon \\
T &\rightarrow FT' \\
T' &\rightarrow *FT' \mid \epsilon \\
F &\rightarrow (E) \mid \mathbf{id}
\end{aligned}
$$

Input: `id + id * id`

NONTERMINAL	INPUT SYMBOL					
	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \mathbf{id}$			$F \rightarrow (E)$		

Moves Made by Predictive Parser

STACK	INPUT	OUTPUT
$\$E$	id + id * id$	
$\$E'T$	id + id * id$	$E \rightarrow TE'$
$\$E'T'F$	id + id * id$	$T \rightarrow FT'$
$\$E'T'\mathbf{id}$	id + id * id$	$F \rightarrow \mathbf{id}$
$\$E'T'$	+ id * id$	
$\$E'$	+ id * id$	$T' \rightarrow \epsilon$
$\$E'T+$	+ id * id$	$E' \rightarrow +TE'$
$\$E'T$	id * id$	
$\$E'T'F$	id * id$	$T \rightarrow FT'$
$\$E'T'\mathbf{id}$	id * id$	$F \rightarrow \mathbf{id}$
$\$E'T'$	* id$	
$\$E'T'F*$	* id$	$T' \rightarrow *FT'$
$\$E'T'F$	id$	
$\$E'T'\mathbf{id}$	id$	$F \rightarrow \mathbf{id}$
$\$E'T'$	$	
$\$E'$	$	$T' \rightarrow \epsilon$
$	$	$E' \rightarrow \epsilon$

Do you have any questions/clarifications?

… few minutes.

Bottom-Up Parsing and LR Parsers

- Shift-reduce parsing attempts to construct a parse tree for an input string beginning at the leaves (bottom) and working up towards the root (top).
- “Reducing” a string w to the start symbol of a grammar.
- At each reduction step a particular substring machining the right side of a production is replaced by the symbol on the left of that production, and if the substring is chosen correctly at each step, a rightmost derivation is traced out in reverse.

Example

- Grammar:

```
S → aABe
A → Abc|b
B → d
```

- Reduction:

abbcde
aAbcde
aAde
aABe
S

$$S \underset{rm}{\Longrightarrow} aABe \underset{rm}{\Longrightarrow} aAde \underset{rm}{\Longrightarrow} aAbcde \underset{rm}{\Longrightarrow} abbcde$$

LR(k) Parsers

- The LR parser is a non-recursive, shift-reduce, bottom-up parser.
- It uses a wide class of context-free grammar which makes it the most efficient syntax analysis technique.
- L: left-to-right scanning of the input
- R: constructing a rightmost derivation in reverse
- k: the number of input symbols of lookahead that are used in making parsing decisions.

- There are three widely used algorithms available for constructing an LR parser:

SLR(1) – Simple LR Parser:
- Works on smallest class of grammar;
- Few number of states, hence very small table;
- Simple and fast construction;

LR(1) – LR Parser:
- Works on complete set of LR(1) Grammar;
- Generates large table and large number of states;
- Slow construction;

LALR(1) – Look-Ahead LR Parser:
- Works on intermediate size of grammar;
- Number of states are same as in SLR(1);

LR Parsing

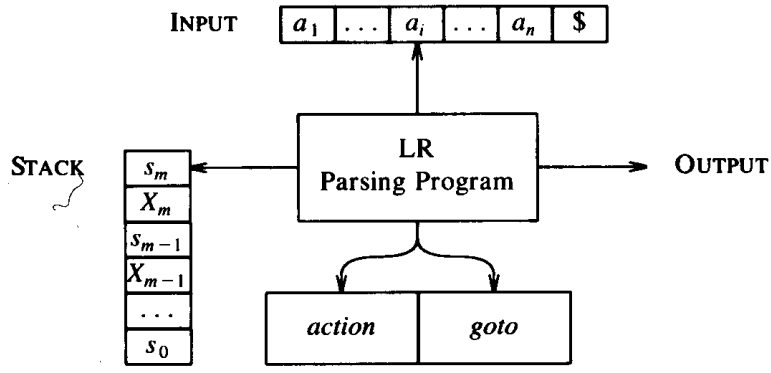
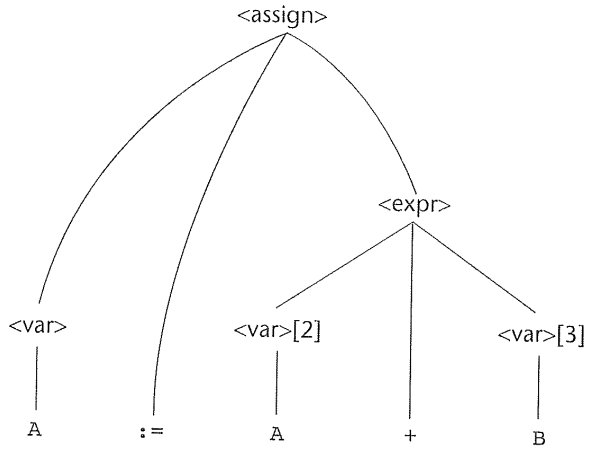


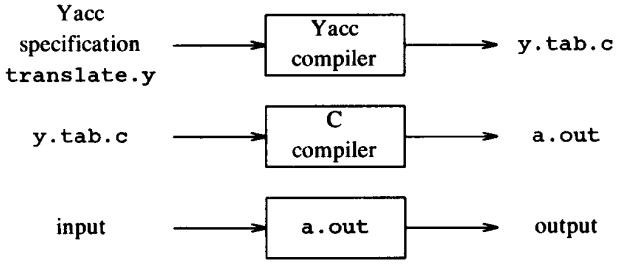
Fig. 4.29. Model of an LR parser.

E.g., A parser tree for A := A + B



- Tools for Syntax Analysis (YACC)

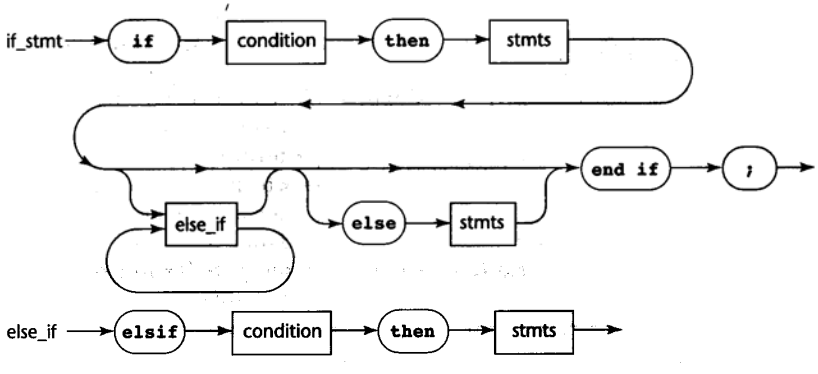
```
declarations
%%
translation rules
%%
supporting C-routines
```



Syntax Graphs

- A graph is a collection of nodes, some of which are connected by lines (edges).
- A directed graph is one in which the lines are directional.
- A parse tree is a restricted form of directed graph.
- Syntax graph is a directed graph representing the information in BNF rules.

Syntax Graph of “if “ statement



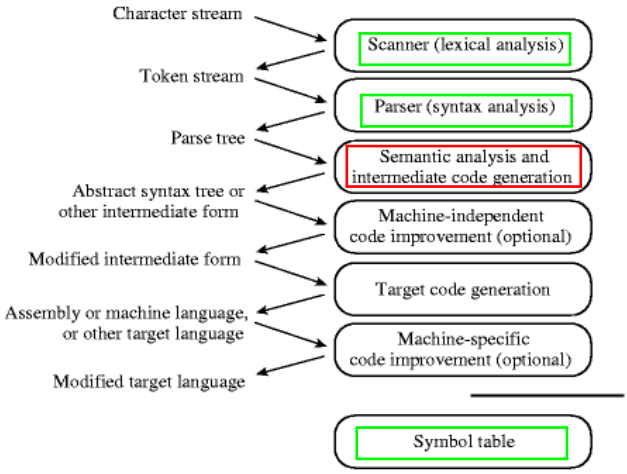
<if_stmt> ⟶ **if** <condition> **then** <stmts> {<else_if>}

[**else** <stmts>] **end if**

<else_if> ⟶ **elsif** <condition> **then** <stmts>

Phases of Compilation

Up to last DS, . . .



What's next?

We would like to:

- perform additional checks to increase guarantees of correctness;
- transform the program from the source language into the target one, and according to precisely defined semantic rules;

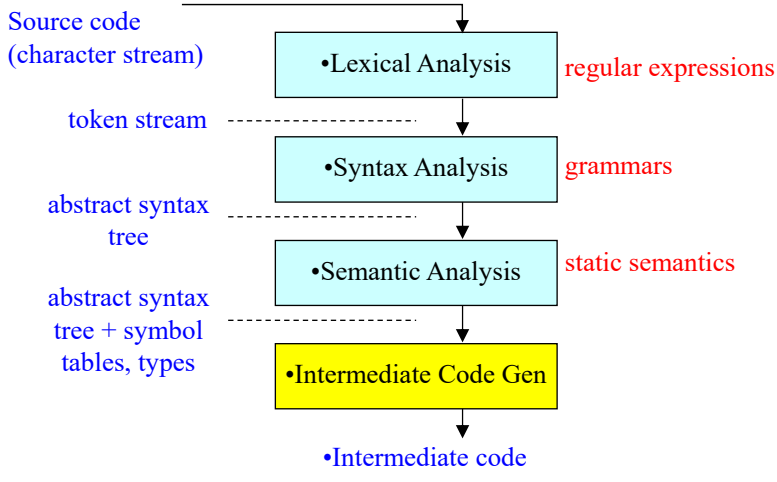
Additional Checks:

There are many additional checks that can be performed to increase correctness of code:

- Coherent (clear idea) usage of variables
  - definition-usage;
  - type;
- Existence of unreachable code blocks, . . .

Here, it is mainly focused on the mechanisms for type checking and generation of intermediate code

Where We Are...



What's next?

- After semantic analysis, we have a number of options to choose from:
  - generate object code directly from the parse.
  - generate intermediate code, and then generate object code from it.
  - generate an intermediate abstract representation, and then generate code directly from it.
  - generate an intermediate abstract representation, generate intermediate code, and then the object code.
- All these options have one thing in common:

  •-- they are all based on syntactic information gathered/aggregated/ processed/verified in the syntactic-semantic analysis.

•**Intermediate representations**

- Synthetize the syntactic information gathered during the parse, generally in the form of a tree or directed graph.
- Intermediate representations enable high-level code optimization.
- Three ways of intermediate representation:
  - Syntax tree
  - Postfix notation
  - Three address code
- In our project, we use an Abstract Syntax Tree (AST) as an intermediate representation.

•**Intermediate code**

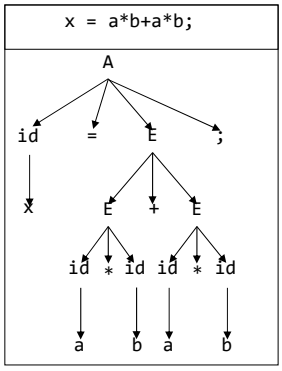
- is a low-level coded (text) representation of the program, directly translatable to object code.
- Intermediate code enables low-level, architecture-dependent optimizations.
- We don't have intermediate code in our project.

For quick reminding …

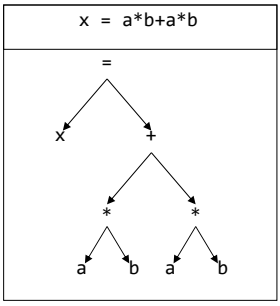
- **Parse trees** (concrete syntax trees) emphasize the grammatical structure of the program, based on the exact concrete syntax of the grammar.
- **Abstract syntax trees** emphasize the actual computations to be performed. They do not refer to the actual non-terminals defined in the grammar, nor to tokens that play no role in defining the translated program, hence their name.
- When implementing an LL parser, there is generally substantial differences between the parse tree and the corresponding abstract syntax tree.

- A parse tree contains <u>grammar-specific nodes</u>, e.g. non-terminal symbols, punctuation, etc. These are not necessary for further processing. Abstract syntax trees <u>do not include them</u>, hence the name *abstract*.

Parse Tree

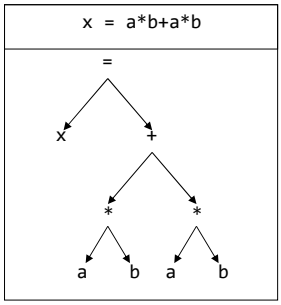


Abstract Syntax Tree

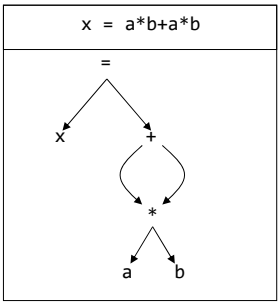


- **Directed acyclic graphs** (DAG) are a relative of syntax trees: they are used to show the syntactic structure of valid programs in the form of a graph with the following restrictions:
  - Edges between nodes can only be **unidirectional**.
  - There cannot be **cycles** in a DAG, i.e. no path can ever lead twice to the same node.
  - Presence of a **root node**.
- For example, in a DAG representation, the nodes for repeated variables and expressions are merged into a single node.
- DAGs are more complex to build and use than syntax trees, but easily allows the implementation of a variety of **optimization** techniques by avoiding redundant operations.

- Abstract Syntax Tree

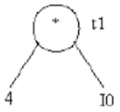


- Directed Acyclic Graph

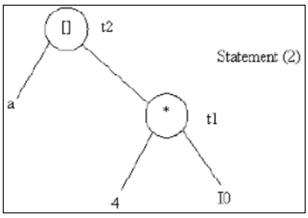


Example: Consider the block of three-address statements:

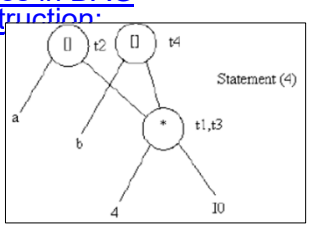
1. t1 := 4* i
2. t2 := a[t1]
3. t3 := 4* i
4. t4 := b[t3]
5. t5 := t2*t4
6. t6 := prod+t5
7. prod := t6
8. t7 := i+1
9. i := t7
10. if i<=20 goto (1)

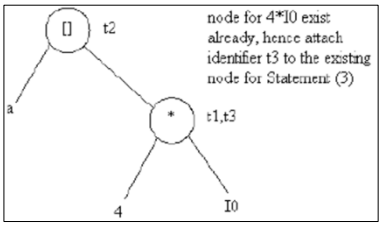


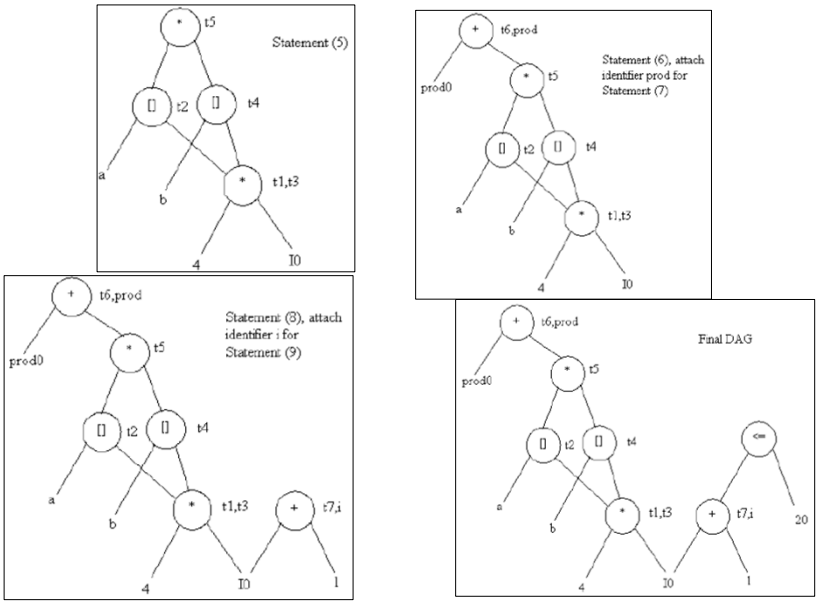
Statement (1)



- Stages in DAG Construction:







Thank You

END.